Foundations of Privacy: Phase 3 Report

Muhammad Bilal Ali*, Aditi Balaji*, Michael Stoneman*

Abstract

Membership Inference Attacks pose significant threats to the privacy of machine learning models by determining whether specific data was used during model training. This work presents a per-token z-score normalization attack against GPT-2 LoRA fine-tuned models, leveraging statistical comparison between a target model and an ensemble of shadow models trained on data from the same distribution. We extract two features for each sample (average per-token cross-entropy loss and perplexity) and compute z-scores that quantify how the target model's behavior deviates from the shadow distribution. A positive z-score indicates that the target model assigns lower loss than expected, providing evidence of membership. Our approach achieves an AUC of 0.9548 and a TPR of 0.5270 at 0.01 FPR on the training set, demonstrating effective membership inference without requiring a separate binary classifier. By using continuous z-scores directly as membership signals, our method simplifies the attack pipeline while maintaining strong performance. This work contributes to understanding privacy vulnerabilities in fine-tuned language models and establishes z-score normalization as an effective calibration technique for membership inference.

1 Introduction and Motivation

Membership Inference Attacks (MIAs) aim to determine whether a specific data sample was used when training a machine learning model. The attacker tries to infer "membership" by querying the model and (if possible) inspecting internal parameters, and by analyzing the statistical differences between the results that are output—the idea being that the model would behave differently, based on whether it's a member sample or a non-member sample. The differences in confidence, loss values, etc., can help the attacker infer membership.

The idea behind MIAs is that models often memorize patterns from their training data, and that this can lead to leakage of personal information or other sensitive information. This becomes a particular concern when they are models that are meant for a sensitive domain, or have been trained on health data, financial information, or other personal information. Additionally, leaking of personal and sensitive information (which can be done via an MIA) would be non-compliant with existing data privacy rules and regulations, such as the GDPR.

Thus, membership inference attacks are a type of privacy attack, and combining them with other attacks such as attribute inference attacks can lead to successful training data extraction. In a world where ML use is ubiquitous, it is all the more important to understand how these attacks work—this would help us understand how they can be effectively prevented.

2 Related Work

Shokri et al. introduced the idea of shadow model training, where multiple shadow models are trained on data which is created a) using black-box access to the target model to synthesize this data, or b) using statistics about the population from which the target's training dataset was drawn, or c) by assuming that the adversary has access to a potentially noisy version of the target's training dataset. The shadow models were then trained on this data to simulate the target model's behavior and then train a binary classifier which distinguished members from non-members [1].

Carlini et al. advanced the field by establishing their Likelihood Ratio Attack (LiRA), which emphasizes evaluation using true positives at low false positive rates and achieves better performance than previous evaluation metrics such as computing likelihood ratios under Gaussian assumptions [2].

Recent work has introduced computationally effective alternatives to shadow model training. Bertran et al. proposed scalable membership inference attacks via quantile regression, demonstrating that a single quantile regression model can achieve performance competitive with multiple shadow models while requiring less computation. They additionally identified pinball loss as a key target objective, wherein methods which produce thresholds minimizing pinball loss, constituted the most successful attacks [3].

Shi et al. introduced min-k% probability, a method specifically designed for large language models that focus on tokens with the least probability in a sequence. Their key insight is that non-members are more likely to contain outlier words with unusually low probabilities compared with member samples. By selecting k% of tokens with lowest log likelihood and averaging their scores, min-k% prob achieves improved attack performance on LLMs without requiring reference models or shadow training [4].

Despite these advances, existing membership inference attacks face several key limitations. Shadow model-based approaches like those of Shokri et al. and LiRA require high computational costs, involving training multiple models with the same architecture as the target and, in many cases, additional binary classifiers or reference models to estimate membership distributions. This makes them computationally prohibitive for large-scale models. Quantile regression methods reduce this burden but still require training separate regression models and make distributional assumptions that may not hold universally. Min-K% and similar score-based methods, while efficient, are sensitive to hyperparameter selection (such as the choice of k) and require careful tuning for different model architectures and datasets.

Our work addresses these computational and complexity limitations by using z-score normalization with shadow models to produce continuous membership scores directly, without training additional classifiers or regression models.

3 Methodology

3.1 Diagrammatic Overview

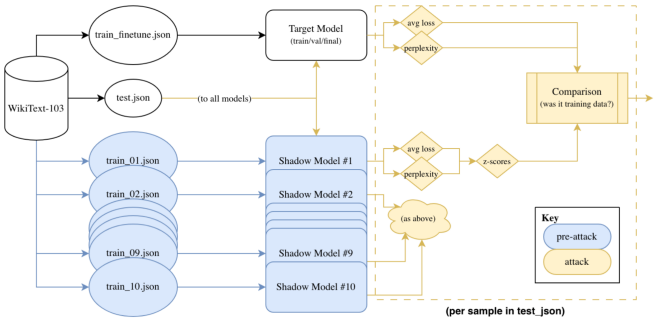


Figure 1: Methodology Overview

3.2 Training Data

The training data set we were provided with contained 10k samples drawn from the same distribution that were used to train the three target models: train, val and final. The membership labels for the train target model were provided to us for us to arrive at the scoring function,

and for us to simulate member and non-member behavior. The shadow models being trained on data from the same distribution meant that there are similarities between the shadow models and the target model—identifying and quantifying that property is what helped us carry out our membership inference attack.

3.3 Shadow Data Split

Shadow datasets were constructed by sampling 10,000 sentences from WikiText-103 (raw) for each shadow model. The following preprocessing was applied:

- Cleaning and removal of corrupted or malformed text.
- Filtering to ensure at least 25 tokens per sample.
- A different random seed for each shadow model to produce diverse sample sets.

This simulated the statistical conditions under which the target GPT-2 model was trained.

3.4 Shadow Models

The shadow models we built were based off GPT-2 LoRA models, and were trained to emulate the target model's architecture and training process as closely as possible. We used the same fine-tuning configuration parameters for the target models: a GPT-2 base model; LoRA with $r = 32$, $\alpha = 64$, and dropout = 0.05; 3 epochs; a batch size of 8 (we reduced this from 50 due to memory constraints), a learning rate of 2e-4 and a weight decay of 0.01.

The seeds we used: 100, 101, 102, 103, 104, 105, 106, 107, 359, 1114.

3.5 Extracting Shadow Feature Scores

The attack used per-token cross-entropy losses.

For each token x_t:

$$L_t = -\log P(x_t | x_{<t}) \tag{1}$$

The average loss across valid tokens was:

$$\text{avgloss}(x) = \frac{1}{T} \sum_{t=1}^{T} L_t \tag{2}$$

The perplexity was:

$$\text{ppl}(x) = \exp(\text{avgloss}(x)) \tag{3}$$

These features were extracted for both the target model and all shadow models.

3.6 Attack Model

Our membership inference attack was based on a statistical comparison between the target model and a collection of shadow models trained on data drawn from the same distribution, using the extracted features.

3.6.1 Shadow Statistics

For each feature $f \in \{\text{avgloss}, \text{ppl}\}$, we aggregated the corresponding values from N shadow models. For each sample x, the shadow mean and standard deviation were:

$$\mu_f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x) \tag{4}$$

$$\sigma_f(x) = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (f_i(x) - \mu_f(x))^2} \tag{5}$$

3.6.2 Z-Score Membership Signal

We compared the target model's behavior to the shadow distribution using a z-score:

$$z_f(x) = \frac{\mu_f(x) - f_{\text{target}}(x)}{\sigma_f(x) + \epsilon} \tag{6}$$

where $\epsilon = 10^{-6}$ ensures numerical stability.

A positive z-score indicates that the target model assigned a lower loss (or lower perplexity) than expected under the shadow distribution—evidence that the sample may have been part of the target model's training data.

3.7 Final Membership Score and Documentation of Prediction

The final score for each sample is obtained by averaging the two feature z-scores:

$$\text{score}(x) = \frac{z_{\text{loss}}(x) + z_{\text{ppl}}(x)}{2} \tag{7}$$

This score was a continuous-valued membership estimate: higher values indicate higher likelihood of membership. No classifier was trained, and no thresholding was applied in the val or final phases. This could be done to improve the score (phase 3 work, potentially). The raw continuous scores were written directly to `prediction.csv` for the val and final phases respectively.

These files were then uploaded onto Codabench, and the AUC and TPR @ 0.01 FPR were generated for the val (validation) and final phases.

3.8 Novelty

The intuition behind our method is that the shadow models provide a baseline for how we expect the target model to behave when provided non-member data. If the target model's behavior deviates sufficiently from this baseline, we can assume that the sample is member data.

4 Experimental Results

4.1 Evaluation Method and Visualization Result

Results (Train):

- AUC Score: 0.9548
- TPR @ 0.01 FPR: 0.6230 (62.3%)

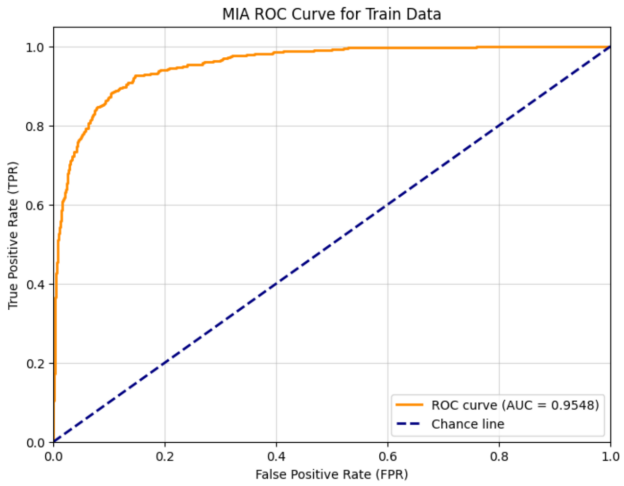


Figure 2: ROC Curve For Train

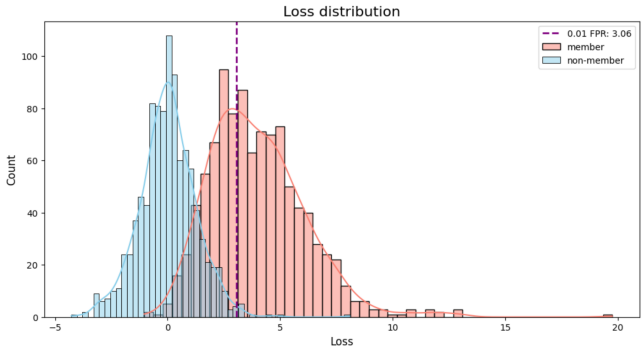


Figure 3: Membership Loss Split

4.2 Ablation Studies

We used 6 shadow models in phase 2 and achieved a TPR of 0.5270 at 0.01 FPR in the train phase. When we used 10 shadow models in phase 3, we got a TPR of 0.6230 in the train phase. We evaluated perplexity and average loss to measure their contribution to the overall attack performance. Combining these features by averaging their results, we got an AUC value of 0.9548.

There are certain limitations with cross entropy loss. Cross-entropy loss measures the model's prediction accuracy on a per-token basis, but it may not fully capture the complex memorization patterns exhibited by language models. Also, cross-entropy loss can be influenced by sample difficulty and domain-specific characteristics; naturally rare or complex tokens will have higher loss regardless of membership status, potentially confounding the membership signal.

Perplexity, being the exponential of average loss, provides a non-linear transformation that amplifies differences between member and non-member samples, making the membership signal more pronounced. Combining both raw loss and its exponential transformation, our approach captures both the linear and non-linear aspects of the model's predictive behavior, resulting in more robust and discriminative membership inference.

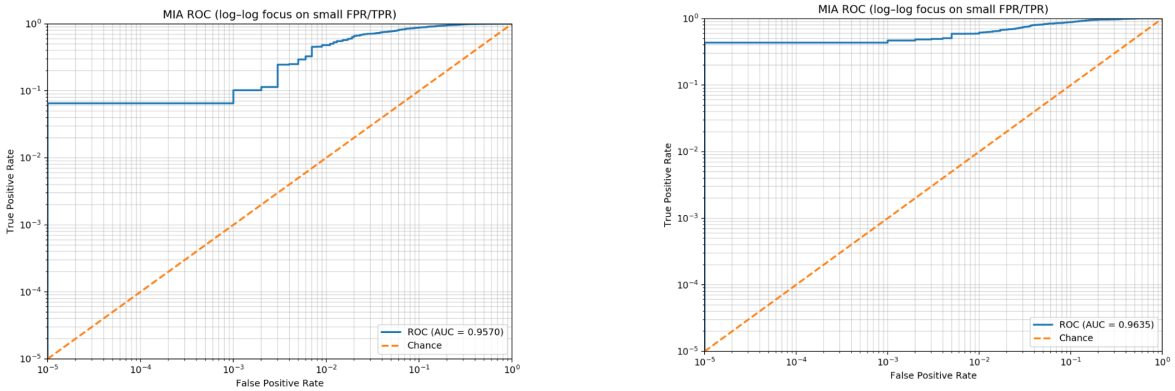
Additional features would require more shadow model computations and potentially more complex aggregation schemes, and although might result in improvements, would not be worth the computational trade-offs.

Can the experiments or demo be reproduced from the information given in the report? Yes.

5 Quality of Results

Running our attack on the target model GPT2 (LoRA 32) gave us the following results:

Against the validation set, we achieved an AUC score of 0.9570, and a TPR of 0.48 at 0.01 FPR with 6 shadow models (Graph 1). Furthermore, when running the attack against the test set, we attained an AUC score of 0.9635, and a TPR of 0.592 at 0.01 FPR with 10 shadow models (Graph 2).



(a) Validation Set Results (6 shadow models)

(b) Final Set Results (10 shadow models)

Figure 4: ROC Curves for Validation and Test Sets

6 Individual Contributions

Bilal wrote the abstract and related work sections, analyzed experimental results, and contributed to the code. Aditi wrote the introduction and methodology sections, worked on code implementation, and finalized the ablation studies section. Michael designed and documented the methodology, implemented core code components, and evaluated result quality and novelty. All team members more or less evenly collaborated on project planning, debugging, and report writing.

References

[1] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," 2017.

[2] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," 2022.

[3] M. Bertran, S. Tang, A. Roth, M. Kearns, J. H. Morgenstern, and S. Z. Wu, "Scalable membership inference attacks via quantile regression," 2023.

[4] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, "Detecting pretraining data from large language models," 2024.

Appendix: Code

```
# Hyperparameters
BATCH_SIZE = 50
BLOCK_SIZE = 512  # Maximum length: 512
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
PHASE = "val"  # Options: "train", "val", "final"

# Path variables
ROOT = Path("/content/18734-17731_Project_Phase2_3")

DATA_DIR = ROOT / "data" / PHASE
TARGET_MODEL_DIR = ROOT / "models" / PHASE / "gpt2_3_lora32_adamw_b8_lr2"
SHADOW_MODELS_ROOT = ROOT / "models" / "shadow"

SHADOW_DATA_ROOT = ROOT / "data" / "shadow"
LABEL_PATH = DATA_DIR / "test_label.json"
PREDICTION_PATH = DATA_DIR / "prediction.csv"
TEST_PATH = DATA_DIR / "test.json"


# Load test data.
test_items = read_json(TEST_PATH)
test_dataset = Dataset.from_dict({"text": test_items})

# Initialize tokenizer.
tokenizer = AutoTokenizer.from_pretrained(TARGET_MODEL_DIR, use_fast=True)

# Configure tokenizer padding.
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

# Tokenize the test data using a previously defined function.
test_dataset = tokenize_dataset(test_dataset, tokenizer, BLOCK_SIZE)

# Create DataLoader.
dataloader_test = DataLoader(test_dataset, batch_size=BATCH_SIZE)


"""
Per-Token Membership Inference Attack
- avg_loss: Average loss across all tokens (overall confidence)
- perplexity: exp(avg_loss)
- Z-score normalization using shadow model distribution
"""

import numpy as np
import torch
from pathlib import Path
from torch.utils.data import DataLoader
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
from typing import Tuple, List
import torch.nn.functional as F


@torch.no_grad()
def extract_per_token_losses(
    model: AutoModelForCausalLM,
    dataloader: DataLoader,
    device: torch.device
) -> Tuple[np.ndarray, np.ndarray]:
```

```
    """
    Extract per-sample features: avg_loss and perplexity
    - avg_loss = (1/T) * sum(L_t) for all T tokens
    - perplexity = exp(avg_loss)

    Returns:
        Tuple of (avg_losses, perplexities) as numpy arrays
    """
    avg_losses = []
    perplexities = []
    model.eval()

    for batch in dataloader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Process each sample individually
        for i in range(input_ids.shape[0]):
            single_input = input_ids[i:i+1]
            single_labels = labels[i:i+1]
            single_mask = attention_mask[i:i+1]

            # Forward pass to get logits
            outputs = model(
                input_ids=single_input,
                attention_mask=single_mask
            )

            logits = outputs.logits  # Shape: (1, seq_len, vocab_size)

            # Shift for causal LM: predict token t using tokens 1...t-1
            shift_logits = logits[:, :-1, :].contiguous()
            shift_labels = single_labels[:, 1:].contiguous()
            shift_mask = single_mask[:, 1:].contiguous()

            # Compute per-token cross-entropy loss
            # L_t = -log P(x_t | x_1,...,x_{t-1})
            loss_fct = torch.nn.CrossEntropyLoss(reduction='none')
            per_token_loss = loss_fct(
                shift_logits.view(-1, shift_logits.size(-1)),
                shift_labels.view(-1)
            )

            # Reshape back to (batch=1, seq_len-1)
            per_token_loss = per_token_loss.view(1, -1)

            # Only average over valid (non-padded) tokens
            valid_tokens = shift_mask.bool()
            num_valid = valid_tokens.sum().item()

            if num_valid > 0:
                # avg_loss = (1/T) * sum(L_t) over valid tokens
                avg_loss = (per_token_loss * valid_tokens).sum().item() /
                    num_valid
            else:
                avg_loss = 0.0

            avg_losses.append(avg_loss)
            perplexities.append(np.exp(avg_loss))

    return np.array(avg_losses), np.array(perplexities)
```

```python
def load_lora_model(model_dir: Path, device: torch.device) ->
    AutoModelForCausalLM:
    """Load a LoRA fine-tuned model"""
    base_model = AutoModelForCausalLM.from_pretrained("gpt2")
    model = PeftModel.from_pretrained(base_model, str(model_dir))
    model = model.merge_and_unload()
    model.to(device)
    return model


@torch.no_grad()
def per_token_zscore_attack(
    target_model_dir: Path,
    shadow_models_root: Path,
    dataloader: DataLoader,
    device: torch.device,
    epsilon: float = 1e-6,
) -> np.ndarray:
    """
    For each test sample x and feature f (avg_loss or perplexity):
    1. Compute f_target(x) from target model
    2. Compute f_shadow_i(x) for each shadow model i
    3. Calculate:
       mu_f(x) = (1/N) * sum(f_shadow_i(x)) for all N shadows
       sigma_f(x) = std(f_shadow_1(x), ..., f_shadow_N(x))
       z_f(x) = (mu_f(x) - f_target(x)) / (sigma_f(x) + epsilon)

    A positive z_f(x) means target loss is LOWER (better) than shadow average
    by more than one std dev -> evidence that x is a member

    Args:
        target_model_dir: Path to target model
        shadow_models_root: Path to shadow models directory
        dataloader: Test data
        device: Compute device
        epsilon: Small constant for numerical stability (10^-6)

    Returns:
        Membership scores (higher = more likely member)
    """

    print("\n" + "="*80)
    print("PER-TOKEN Z-SCORE NORMALIZATION ATTACK")
    print("="*80)

    # STEP 1: Extract Target Features
    print("\n Step 1: Extracting per-token features from TARGET model")
    target_model = load_lora_model(target_model_dir, device)

    target_avg_loss, target_perplexity = extract_per_token_losses(
        target_model, dataloader, device
    )

    print(f"  Extracted features for {len(target_avg_loss)} samples")
    print(f"  Mean avg_loss: {target_avg_loss.mean():.4f}")
    print(f"  Mean perplexity: {target_perplexity.mean():.2f}")

    # Cleanup
    del target_model
    if device.type == 'cuda':
        torch.cuda.empty_cache()
```

```
    # STEP 2: Extract Shadow Features
    print("\n Step 2: Extracting per-token features from SHADOW models")

    shadow_model_dirs = [
        d for d in shadow_models_root.iterdir()
        if d.is_dir() and "shadow_model" in d.name
    ]

    if len(shadow_model_dirs) == 0:
        print("ERROR: No shadow models found!")
        return np.zeros(len(target_avg_loss))

    print(f" Found {len(shadow_model_dirs)} shadow models")

    all_shadow_avg_losses = []
    all_shadow_perplexities = []

    for i, shadow_dir in enumerate(shadow_model_dirs):
        print(f"\n Processing shadow {i+1}/{len(shadow_model_dirs)}: {
            shadow_dir.name}")

        try:
            shadow_model = load_lora_model(shadow_dir, device)

            shadow_avg_loss, shadow_perplexity = extract_per_token_losses(
                shadow_model, dataloader, device
            )

            all_shadow_avg_losses.append(shadow_avg_loss)
            all_shadow_perplexities.append(shadow_perplexity)

            print(f"    * Mean avg_loss: {shadow_avg_loss.mean():.4f}")
            print(f"    * Mean perplexity: {shadow_perplexity.mean():.2f}")

            del shadow_model
            if device.type == 'cuda':
                torch.cuda.empty_cache()

        except Exception as e:
            print(f"Error: {e}")
            continue

    if len(all_shadow_avg_losses) == 0:
        print("\n ERROR: Failed to load any shadow models!")
        return np.zeros(len(target_avg_loss))

    # STEP 3: Z-Score Normalization
    print("\n Step 3: Computing Z-score normalization.")

    # Stack shadow features: shape (n_samples, n_shadows)
    shadow_losses_matrix = np.stack(all_shadow_avg_losses, axis=1)
    shadow_perps_matrix = np.stack(all_shadow_perplexities, axis=1)

    # For each sample x and feature f:
    # mu_f(x) = mean of shadow values
    mu_loss = shadow_losses_matrix.mean(axis=1)  # Shape: (n_samples,)
    mu_perp = shadow_perps_matrix.mean(axis=1)

    # sigma_f(x) = std of shadow values
    sigma_loss = shadow_losses_matrix.std(axis=1)  # Shape: (n_samples,)
    sigma_perp = shadow_perps_matrix.std(axis=1)

    # z_f(x) = (mu_f(x) - f_target(x)) / (sigma_f(x) + epsilon)
```

```
    z_loss = (mu_loss - target_avg_loss) / (sigma_loss + epsilon)
    z_perp = (mu_perp - target_perplexity) / (sigma_perp + epsilon)

    print(f"  Z-score statistics:")
    print(f"  z_loss:  mean={z_loss.mean():.4f}, std={z_loss.std():.4f}")
    print(f"  z_perp:  mean={z_perp.mean():.4f}, std={z_perp.std():.4f}")

    # STEP 4: Combine Features
    print("\n Step 4: Combining features into final score.")

    scores = (z_loss + z_perp) / 2
    print(f"  Generated {len(scores)} membership scores")
    print(f"  Mean: {scores.mean():.4f}")
    print(f"  Std: {scores.std():.4f}")
    print(f"  Min/Max: [{scores.min():.4f}, {scores.max():.4f}]")

    # STEP 5: Analysis
    print("\n Step 5: Attack analysis...")

    # How many samples have positive z-scores? (predicted as members)
    positive_count = (scores > 0).sum()
    print(f"  Samples with positive z-score: {positive_count}/{len(scores)}")
    print(f"  (These are predicted as members)")

    # Distribution analysis
    print(f"\n  Z-score distribution:")
    print(f"    < -1.0 (strong non-member): {(scores < -1.0).sum()}")
    print(f"    -1.0 to 0 (weak non-member): {((scores >= -1.0) & (scores < 0))
        .sum()}")
    print(f"    0 to +1.0 (weak member): {((scores >= 0) & (scores < 1.0)).sum
        ()}")
    print(f"    > +1.0 (strong member): {(scores > 1.0).sum()}")

    print("\n" + "="*80)
    print("PER-TOKEN Z-SCORE ATTACK COMPLETE")
    print("="*80)
    print("\n Interpretation:")
    print("  Positive score: Target loss < shadow average means likely MEMBER")
    print("  Negative score: Target loss > shadow average means likely NON-
        MEMBER")
    print("  Score magnitude: How many std devs from shadow distribution")

    return scores

# Run the attack
scores_test = per_token_zscore_attack(
    target_model_dir=TARGET_MODEL_DIR,
    shadow_models_root=SHADOW_MODELS_ROOT,
    dataloader=dataloader_test,
    device=DEVICE
)
```